BOWCAR + HUSKYLENS

바우카 + 허스키렌즈 강사용 설치·실습 매뉴얼

Python 3.11 / ATmega328P·UNO 호환 보드
설치부터 기본 주행, 센서 활용, 인식 연동까지

이 안내서는 처음 수업을 준비하는 강사가 바우카의 기본 주행·센서와 허스키렌즈 연동을 순서대로 확인하는 방법을 설명합니다. 상세 함수와 인식 모드는 README, 소프트웨어 검사 기록은 VERIFICATION.txt를 참고하세요.

주행 전 배터리 전원 ON
이번 실습 장치의 배터리는 3.7V 리튬폴리머입니다. USB는 업로드·시리얼 연결에 사용하며, USB 단독을 주행용 전원으로 전제하지 않습니다.

목차

1. 전원과 실행 방식부터 확인하기 2

2. PC 설치 3

3. 포트와 공통 실행 옵션 4

4. 기본 주행·센서 실습 6

5. 허스키렌즈 연결과 인식 실습 8

6. 반복 구조와 조건문 이해하기 9

7. 로그 보기와 문제 대응 10

8. 권장 수업 순서 11

9. 확인된 사실과 검증 경계 12

부록. 허스키렌즈 함수와 인식 모드 13

공개 저장소와 예제 받기 / 온라인 매뉴얼 원문

1. 전원과 실행 방식부터 확인하기

배터리와 USB의 역할

주행 실습에는 바우카에 맞는 배터리를 연결하고 배터리 전원을 ON으로 둡니다. 이번 실습 장치에서 사용자가 확인한 배터리는 3.7V 리튬폴리머입니다. USB는 업로드와 시리얼 모니터 연결에 사용합니다. USB 단독 연결을 주행용 전원으로 안내하지 않습니다.

이번 사용자 확인에서는 배터리 전원을 ON으로 두면 정상 동작했고, PC USB만 연결했을 때는 정상으로 확인한 보드에서도 재시작이 발생했습니다. 이 관찰만으로 보드 손상이나 전원부 손상을 확정할 수 없습니다. 실제 전원 경로, 동작 중 전압과 재시작 원인은 측정으로 확인하지 않았습니다.

제품에 지정된 배터리 정격·커넥터·극성을 지키고, 충전은 제품 안내에 따릅니다. USB가 연결되어 있다는 이유만으로 배터리가 충전 중이라고 판단하지 않습니다.

연결 상태	수업에서의 사용
USB 연결	프로그램 업로드, 포트 확인, 시리얼 로그 보기
지정 배터리 연결·전원 ON	주행·모터 실습의 전원 조건
USB 단독	주행 정상 여부를 판정하는 기준으로 사용하지 않음

업로드가 끝나면 프로그램이 바로 시작할 수 있습니다. 처음 주행 프로그램을 올릴 때는 바퀴를 바닥에서 띄운 상태로 방향과 정지 구간을 확인한 뒤 내려놓습니다.

Upload와 Live의 차이

방식	Python과 보드의 역할	이 저장소에서의 사용
Upload / 코드 생성	PC의 Python이 Arduino `.ino`를 생성. 컴파일·업로드 후에는 MCU에서 독립 실행	모든 실습 예제와 허스키렌즈 확장
Live / PC 연결 제어	PC Python이 실행 중 보드와 통신하며 명령·데이터를 주고받는 방식	기본 `bowcar`의 별도 기능. 이 저장소의 허스키렌즈 확장은 Live 인터페이스를 제공하지 않음

허스키렌즈 결과 함수는 Python에서 실시간 좌표 숫자를 반환하는 함수가 아닙니다. MCU에서 평가할 **C++ 표현식 문자열**을 반환합니다. PC에서 `print(car.get_target_x())`로 좌표를 읽는 대신 업로드한 프로그램의 `debug_huskylens()`와 시리얼 모니터를 사용합니다.

PC에서 인식 데이터를 직접 받아 분석하는 수업은 MCU 시리얼 출력과 Python 수신 코드를 별도로 구성해야 합니다.

2. PC 설치

준비물

- Python을 설치할 수 있는 PC와 USB 데이터 케이블
- 바우카와 해당 보드에 맞는 배터리(이번 사용자 확인 장치: 3.7V 리튬폴리머)
- 허스키렌즈 실습 시 HUSKYLENS 1(SEN0305)와 연결 케이블
- 거리 실습 시 보드에 맞는 초음파 센서
- 초기 패키지·보드 코어·라이브러리 다운로드를 위한 인터넷 연결

Python 버전

수업용 환경은 Python 3.11을 권장합니다. `bowcar==1.3.4`의 PyPI 메타데이터에는 Python `>=3.8`이 명시되어 있습니다. 저장소에는 Python 3.8 문법 호환 검사와 Python 3.11 소프트웨어 검사 기록이 있습니다. 설치 허용 버전과 검증된 실행 환경은 구분하며, 모든 Python 3.8 이상 버전에서 실물 동작을 확인한 것은 아닙니다.

Python 설치 후 터미널에서 확인합니다.

```
python --version
python -m pip --version
```

여러 Python이 설치되어 있다면 수업에서 사용할 3.11 환경을 선택합니다. 아래 명령의 `python`은 그 환경을 가리켜야 합니다. Windows에서 Python 실행기가 설치되어 있으면 `py -3.11`로 버전을 지정할 수도 있습니다.

저장소 받기와 의존성 설치

GitHub 저장소의 **Code → Download ZIP**으로 받아 압축을 풀거나 Git으로 받습니다.

```
git clone https://github.com/mkidkm/bowcar_with_huskeylens.git
cd bowcar_with_huskeylens
python -m pip install -r requirements.txt
```

ZIP으로 받았다면 `requirements.txt`가 보이는 폴더에서 설치 명령을 실행합니다. 이 명령은 외부 Python 의존성 `bowcar==1.3.4`를 설치합니다. Arduino CLI와 보드 코어까지 자동 설치하지는 않습니다.

다른 프로젝트와 의존성을 나누려면 가상 환경을 사용할 수 있습니다. Windows PowerShell에서는 활성화 없이도 다음과 같이 실행할 수 있습니다.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

가상 환경을 만들었다면 뒤의 `python` 명령 대신 `.\.venv\Scripts\python.exe`를 사용합니다.

Arduino CLI와 UNO 코어

Arduino CLI 공식 설치 안내에 따라 CLI를 설치하고 PATH에 등록합니다. 새 터미널에서 버전을 확인한 뒤 Arduino AVR Boards 코어를 설치합니다.

```
arduino-cli version
arduino-cli core update-index
arduino-cli core install arduino:avr
arduino-cli core list
```

현재 예제의 보드 선택은 `arduino:avr:uno`입니다. **ATmega328P / Arduino UNO 호환 보드·부트로더** 기준이며 이 선택에는 **MiniCore가 필요하지 않습니다.** 실제 보드의 칩·클럭·부트로더가 다른 경우에는 그 보드에 맞는 FQBN을 사용해야 합니다.

`Platform MiniCore:avr not found`가 나오면 실행 중인 파일과 보드 설정을 확인합니다. 예전 사본이 MiniCore를 선택하고 있는지 점검하고, 현재 UNO 호환 보드 실습은 아래의 `--fqbn arduino:avr:uno`를 사용합니다. 칩 이름만으로 부트로더 호환성을 확정할 수는 없습니다.

3. 포트와 공통 실행 옵션

USB 데이터 케이블로 바우카를 연결하고 포트를 확인합니다.

```
arduino-cli board list
```

Windows의 `COM11`은 지정 방법을 보여 주는 예시입니다. 실제 연결 포트를 사용하세요. 포트가 나타나지 않으면 데이터 케이블·연결 상태를 확인하고, 사용한 USB-시리얼 칩에 맞는 드라이버는 제품 안내에 따라 설치합니다.

`examples/`의 공통 규칙

기본 주행·센서 예제와 `examples/`의 허스키렌즈 예제는 옵션 없이 실행하면 `.ino` 생성만 수행합니다. 이 단계에서는 보드 프로그램이 바뀌지 않습니다.

```
# 1. Arduino 스케치 생성만
python examples/basic_drive.py

# 2. 컴파일만
python examples/basic_drive.py --compile --fqbn arduino:avr:uno

# 3. 컴파일·업로드, 포트 자동 선택
python examples/basic_drive.py --upload --fqbn arduino:avr:uno

# 4. 컴파일·업로드, 포트 직접 지정
python examples/basic_drive.py --upload --fqbn arduino:avr:uno --port COM11
```

다른 예제도 파일명만 바꾸면 같은 옵션을 사용합니다.

옵션	사용 목적
없음	generated/에 .ino 생성만
--compile	보드에 쓰지 않고 컴파일만
--upload	컴파일 후 보드에 업로드
--fqbn arduino:avr:uno	UNO 호환 보드 설정. 컴파일·업로드 시 지정
--port COM11	실제 USB 포트 직접 지정. 생략하면 Arduino/CH340 포트 자동 탐색
--reverse-left	왼쪽 모터 극성 반전
--reverse-right	오른쪽 모터 극성 반전
--output	생성 스케치 폴더 지정

자동 탐색은 해당 포트가 하나일 때 선택합니다. 없거나 여러 개이면 실제 포트를 직접 지정하세요. 모터 극성은 바퀴를 들어 올린 상태에서 확인합니다.

```
python examples/basic_drive.py --upload --fqbn arduino:avr:uno --port COM11 --reverse-left
```

루트의 색·얼굴 파일은 실행 방식이 다름

upload_color_neopixel.py와 upload_face_follow.py는 **직접 실행하면 컴파일·업로드**합니다. 이 두 파일은 upload_settings.py의 PORT, FQBN, 모터 극성을 읽으며 공통 --upload 옵션을 받는 예제 파일이 아닙니다.

```
# upload_settings.py
PORT = None             # 자동 탐색. 직접 지정하려면 "COM11"
FQBN = "arduino:avr:uno"
LEFT_POLARITY = 1         # 반대로 돌면 -1
RIGHT_POLARITY = 1
```

examples/의 공통 옵션은 이 설정 파일을 자동으로 읽지 않습니다. examples/ 파일에는 --fqbn, 필요한 경우 --port·극성 옵션을 직접 지정합니다.

4. 기본 주행·센서 실습

실습 1: 전원 확인 - power_check.py

먼저 **idle 모드**로 모터를 정지시킨 상태의 동작을 확인합니다. 기본값이 idle이므로 --mode를 생략해도 됩니다.

```
python examples/power_check.py --mode idle --compile --fqbn arduino:avr:uno
python examples/power_check.py --mode idle --upload --fqbn arduino:avr:uno --port COM11
```

프로그램이 시작되면 BOOT를 한 번 출력하고 빨강·파랑 LED가 함께 빠르게 3회 깜빡입니다. idle에서는 모터가 정지하고, 파랑 LED가 500ms 켜짐·500ms 꺼짐을 반복하며 IDLE 로그를 출력합니다. 배터리 전원 ON 조건에서 이 동작과 로그를 관찰합니다. 구동을 포함한 비교는 바퀴를 들어 올린 상태에서 **drive 모드를 명시**하여 진행합니다.

```
python examples/power_check.py --mode drive --upload --fqbn arduino:avr:uno --port COM11
```

drive 모드는 아래 기본 주행의 동작 순서를 재사용합니다. 이 예제의 로그는 프로그램 시작과 동작을 관찰하는 자료입니다. 공급 전압을 직접 측정하거나 재시작 원인을 부품 단위로 진단하지는 않습니다. USB 단독 결과와 배터리 ON 결과를 섞어서 정상·고장을 판정하지 않습니다.

실습 2: 기본 주행 - basic_drive.py

```
python examples/basic_drive.py --upload --fqbn arduino:avr:uno --port COM11
```

기본 PWM 값은 **100**, 각 방향으로 구동하기 전 정지 구간은 **300ms**입니다. 전진 1초 → 후진 1초 → 왼쪽 회전 0.5초 → 오른쪽 회전 0.5초의 각 동작 전에 정지하고, 마지막에는 2초 동안 정지한 뒤 반복합니다. 로그의 STEP은 동작 시작, DONE은 해당 대기 구간 완료를 표시합니다.

PWM 값은 모터에 주는 출력 크기이며 일정한 cm/s를 뜻하지 않습니다. 처음에는 바퀴를 들어 올려 좌우와 극성을 확인하고, 파일의 SPEED, PAUSE_MS와 각 동작 시간을 조금씩 조정합니다.

모터를 반대로 돌리는 명령 사이에 정지 구간을 유지하세요. 반복 중 문제가 생기면 배터리 전원을 OFF로 내려 구동을 멈추고 점검합니다.

실습 3: 센서와 LED - sensor_leds.py

```
python examples/sensor_leds.py --upload --fqbn arduino:avr:uno --port COM11
```

이 예제는 **모터를 정지**하고 선택한 센서 입력을 LED 출력과 연결합니다. 기본값은 --sensor button입니다. 조도·소리·라인을 시험하려면 선택 옵션을 지정하여 다시 생성·업로드합니다.

```
python examples/sensor_leds.py --sensor light --upload --fqbn arduino:avr:uno --port COM11
python examples/sensor_leds.py --sensor sound --upload --fqbn arduino:avr:uno --port COM11
python examples/sensor_leds.py --sensor line --upload --fqbn arduino:avr:uno --port COM11
```

어떤 센서를 선택해도 버튼 `u/d/l/r`, 조도 `light`, 소리 `sound`, 좌우 라인 `line_l/line_r`의 **8개 입력값을 모두 250ms 주기로 출력**합니다. 센서를 하나씩 변화시키며 시리얼 값과 LED 반응을 비교합니다.

선택	기본 LED 조건	강사가 확인할 입력 변화
`button`	아래 버튼 d→빨강, 위 버튼 u→파랑	눌렀을 때와 놓았을 때
`light`	값 <500→빨강, >=500→파랑	밝은 상태와 가린 상태의 실제 값
`sound`	값 <=500→빨강, >500→파랑	조용한 상태와 소리를 낸 상태의 실제 값
`line`	왼쪽 값 <500→빨강, 오른쪽 값 <500→파랑	바닥과 라인 위의 값, 좌우 차이

500은 예제의 시작값이며 고정된 환경 기준이 아닙니다. 먼저 두 상태의 값을 기록한 뒤 그 사이 값으로 파일 위쪽의 `LIGHT_THRESHOLD`, `SOUND_THRESHOLD`, `LINE_THRESHOLD`를 조절합니다. 라인 센서는 바닥 색·높이·조명에 따라 값이 달라질 수 있으므로 실제 값이 커지는 조건과 작아지는 조건을 확인하고 필요하면 비교 방향도 바꾸세요.

버튼과 라인 모드는 빨강·파랑 조건을 각각 평가하므로 양쪽 조건이 맞으면 두 LED가 함께 켜질 수 있습니다. 선택하지 않은 센서 값도 로그에 보이지만 LED는 `--sensor`로 고른 조건을 따릅니다. 아날로그 입력은 센서가 연결되지 않아도 항상 0이라는 보장이 없으므로, 미연결 값을 실제 센서 측정값으로 사용하지 않습니다.

실습 4: 거리 조건으로 정지 - `obstacle_stop.py`

초음파 센서가 보드에 맞게 연결된 상태에서 실행합니다.

```
python examples/obstacle_stop.py --upload --fqbn arduino:avr:uno --port COM11
```

거리 측정 전에 모터를 정지하고, 초음파 echo를 최대 25ms 기다려 거리 값을 한 번 저장합니다. 유효한 거리 값이 **20cm보다 클 때** PWM **100**으로 전진하고, 20cm 이하이면 정지합니다. 초음파 센서가 없거나 반사 신호를 받지 못해 **echo 결과가 0이면 정지**합니다. 센서가 없는 상황을 먼 거리로 해석하여 전진하지 않습니다.

판단 후 100ms를 기다리고 다음 측정 전에 다시 정지하므로, 전진 조건에서도 측정 사이에 짧은 정지가 들어갑니다. 이 예제는 거리 입력과 정지 조건을 확인하는 실습용 동작입니다.

초음파 거리와 허스키렌즈 사각형 너비는 서로 다른 값입니다. 얼굴 추종의 120픽셀 정지 기준을 120cm로 해석하지 마세요. 거리 실습은 실제 센서 위치, 물체 방향과 표면을 확인하며 진행합니다.

5. 허스키렌즈 연결과 인식 실습

I2C 배선과 준비

허스키렌즈에서 **General Settings → Protocol Type → I2C**를 선택합니다. 사용 중인 인식 모드에서 대상을 먼저 학습하고 화면의 ID를 확인합니다.

허스키렌즈	UNO 호환 바우카
T / SDA	A4 / SDA
R / SCL	A5 / SCL
+	UNO 연결 기준 5V 전원. 실제 바우카 단자 표기·출력과 카메라 공급 사양 3.3~5.0V 확인
-	보드 GND와 연결, 공통 접지

카메라의 +는 전원 단자이며 모터 제어 GPIO가 아닙니다. **3.7V 배터리 정격과 카메라 연결 단자의 실제 전압은 별개의 확인 항목**입니다. 보드의 지정 연결부를 사용하고 J5 번호만으로 신호를 판단하지 않습니다. 배선·J5 수정 도면 설명은 README의 I2C 연결을 참고하세요.

기존 허스키렌즈 예제 6개

파일	실습 내용·예상 동작
`examples/connection_check.py`	연결·인식 상태 출력, 모터 정지 유지
`examples/follow_object.py`	Object Tracking의 학습 ID1을 추종. 대상 소실·너비 기준에서 정지
`examples/tag_actions.py`	Tag Recognition의 ID1 정지, ID2 왼쪽, ID3 오른쪽, ID4 전진. 작은 ID 우선
`examples/line_follow.py`	Line Tracking의 학습 ID1 화살표 끝점을 따라 조향
`upload_color_neopixel.py`	Color Recognition에서 가장 큰 학습 대상의 ID1/2/3→네오픽셀 빨강/초록/파랑. 모터 정지
`upload_face_follow.py`	Face Recognition의 학습 ID1 x 좌표로 조향·전진. 너비 120픽셀 이상·미검출에서 정지

먼저 연결 확인 예제를 생성·컴파일한 다음 업로드합니다.

```
python examples/connection_check.py
python examples/connection_check.py --compile --fqbn arduino:avr:uno
python examples/connection_check.py --upload --fqbn arduino:avr:uno --port COM11
```

나머지 `examples/`의 허스키렌즈 파일도 같은 옵션을 사용합니다. 루트의 색·얼굴 파일은 설정을 확인한 뒤 실행합니다.

```
# 각각 컴파일·업로드를 수행함. 원하는 프로그램 하나를 선택
python upload_color_neopixel.py
python upload_face_follow.py
```

색 인식은 **학습 ID**에 따라 출력합니다. 빨강·초록·파랑을 순서대로 학습하여 ID1·ID2·ID3인지 확인합니다. 색·얼굴 파일은 전원이 켜질 때 지정 모드로 전환합니다. 연결·모드 응답에 실패하면 정지 상태로 재시도합니다. 이는 통신이 길게 멈춘 모든 상황의 정지를 보장하는 하드웨어 감시 기능은 아닙니다.

허스키렌즈 함수 19개, 지원 모드 7개, 모터를 정지한 간단 예제 2개는 README의 함수 목록과 간단한 사용 예제에 정리되어 있습니다.

6. 반복 구조와 조건문 이해하기

Python 파일은 명령을 한 번 실행해 Arduino 코드를 생성합니다. 아래 코드는 PC에서 즉시 바퀴를 돌리는 코드가 아니라 MCU의 `loop()`에 넣을 동작을 만드는 코드 조각입니다.

```
# 예제에서 준비한 car 객체를 사용하는 코드 조각
car.motor(100, 100)
car.delay(500)
car.motor(0, 0)
car.delay(300)
```

업로드한 MCU는 `loop()`를 반복합니다. Python에 무한 반복문을 추가해서 파일 생성·업로드를 계속 실행할 필요는 없습니다. 한 번만 실행할 내용은 `setup()`에, 계속 판단할 내용은 `loop()`에 들어가는 구조로 생각하세요.

Upload의 센서 결과는 표현식 문자열이므로 생성 코드의 조건문에는 `with car.bif(...)`, `car.belif(...)`, `car.belse()`를 사용합니다. 아래는 허스키렌즈 인식 유무로 LED를 켜는 코드 조각입니다.

```
car.read_huskylens()
with car.bif(car.has_target()):
    car.blue("on")
with car.belse():
    car.blue("off")
```

인식 결과를 사용하기 전에 `read_huskylens()`를 넣습니다. 얼굴·물체의 너비는 픽셀 단위이며 실제 거리를 측정한 값이 아닙니다.

7. 로그 보기와 문제 대응

Arduino IDE의 시리얼 모니터에서 실제 포트와 **9600 baud**를 선택합니다. 업로드할 때는 시리얼 모니터를 닫아 포트 점유를 풀고, 업로드가 끝나면 다시 엽니다.

허스키렌즈 예제의 `debug_huskylens()`는 다음과 같은 상태를 출력합니다.

```
camera=1 target=1 id=1 x=160 y=120 w=60 h=70
```

위 숫자는 로그 형식을 설명하는 **예시 값**입니다.

상태	먼저 확인할 내용
`camera=0`	I2C 설정·배선·전원·카메라 연결과 모드 설정 응답
`camera=1 target=0`	요청은 성공했지만 선택 조건에 맞는 대상 없음. 학습 ID·인식 모드 확인
얼굴 `target=1 id=1 w>=120`	얼굴 너비 정지 기준에 해당
색 `target=1 id>3`	선택한 대상에는 LED 색을 할당하지 않았으므로 꺼짐
좌표가 보이지만 주행 방향이 반대	바퀴를 들어 올려 좌우 모터와 극성 확인

BOOT가 계속 반복될 때

새 기본 예제 4개는 `setup()`에서 BOOT와 양쪽 LED 빠른 3회 깜빡임을 한 번 수행합니다. 시리얼 연결·업로드 직후 한 번 시작하는 것과, 연결을 유지한 상태에서 짧은 간격으로 이 시작 표시가 계속 반복되는 현상을 구분합니다. 반복 시작 표시는 프로그램 재시작을 관찰하는 근거이며 원인 이름을 확정하는 자료는 아닙니다.

1. 구동을 멈추고 배터리 전원을 OFF로 내립니다.
2. 지정된 배터리의 정격·극성·커넥터와 전원 스위치를 확인합니다.
3. `power_check.py`의 idle 프로그램을 올리고 배터리 전원 ON 조건으로 다시 관찰합니다.
4. idle이 안정적이면 바퀴를 들어 올린 상태에서 drive 모드를 시험합니다.
5. 실행한 예제·모드·포트·전원 조건과 반복 로그를 기록합니다. 같은 전원 조건의 비교 자료가 필요합니다.

USB 단독에서도 주행해야 한다고 가정해 코드를 계속 바꾸거나, 이 현상만으로 보드·전원부 손상을 단정하지 않습니다. 배터리 ON 조건에서도 반복되면 실제 전압·전원 경로를 제품 담당자가 확인할 수 있도록 관찰 기록을 전달합니다.

자주 만나는 설치·업로드 오류

오류·증상	확인할 항목
`No module named bowcar`	예제 실행에 쓰는 동일 Python으로 `python -m pip install -r requirements.txt` 실행
`arduino-cli was not found`	CLI 설치·PATH, 새 터미널에서 `arduino-cli version` 확인
`Platform arduino:avr not found`	`arduino-cli core install arduino:avr`와 `core list` 확인
`Platform MiniCore:avr not found`	예전 사본·FQBN 여부 확인. 현재 UNO 실습은 `arduino:avr:uno` 지정
포트 자동 탐색 실패	`board list`로 확인하고 `--port` 또는 `PORT`를 실제 값으로 지정
포트를 열 수 없음	시리얼 모니터·다른 연결 프로그램 닫기, 케이블·포트 재확인
컴파일 성공 후 업로드 실패	실제 보드·부트로더·포트 대조. 컴파일 성공만으로 보드 선택이 입증되지는 않음
NeoPixel 헤더 없음	`arduino-cli lib install "Adafruit NeoPixel"`로 설치

8. 권장 수업 순서

순서	활동	강사가 확인할 결과
1	Python·CLI·보드 코어 설치	버전과 설치 목록 확인
2	스케치 생성과 컴파일	보드에 쓰기 전 생성 파일·컴파일 결과 확인
3	`power_check idle`	배터리 ON, 모터 정지 상태에서 로그 관찰
4	`basic_drive`·극성 조정	띄운 바퀴의 좌우·전후 방향과 정지 구간
5	`sensor_leds`	입력 변화·로그·LED의 관계, 임계값 조정
6	`obstacle_stop`	유효 거리·가까운 거리·미응답의 서로 다른 결과
7	허스키렌즈 연결·학습	I2C 상태, 모드, 대상 ID와 좌표
8	색/태그 LED 활동 후 얼굴·물체·선 추종	정지 조건부터 확인한 뒤 주행 활동

9. 확인된 사실과 검증 경계

항목	근거 종류	확인 범위
배터리 ON이면 정상, USB 단독에서 정상 보드도 재시작	사용자 실물 관찰	이번 시험 조건의 관찰. 전원 경로·전압·원인 측정은 미확인
Python 3.8 문법 호환·Python 3.11 소프트웨어 검사	PC·CI 검사	문법·코드 생성·시험 결과. 모든 Python 버전과 실물 호환 확인은 아님
기존 허스키렌즈 6개 스케치 UNO 컴파일	PC AVR 컴파일	빌드와 정적 메모리 사용량. 업로드·배선·인식·주행은 별도
허스키렌즈 정상·실패·복구와 출력 조건	호스트 모의 시험	코드 분기·프로토콜·버퍼 시험. 카메라·LED 신호를 실물에서 측정한 것은 아님
기본 주행·센서·전원 확인 예제	소프트웨어 검사와 후속 실물 확인 대상	최신 검사 결과는 VERIFICATION.txt 참고. 이 안내서 자체가 실물 시험 결과를 추가하지 않음

제조사 라이브러리의 메모리 할당 실패 처리, 코어별 I2C 타임아웃 지원, 응답 대기 중 오래된 결과 검사에는 알려진 한계가 있습니다. 실습 전 README의 검증 범위와 알려진 한계를 읽고, 수업 준비 기록에는 **PC 검사 결과와 실제 보드 관찰을 별도로** 남기세요.

부록. 허스키렌즈 함수와 인식 모드

이 표는 `HuskyLensBowCar`의 Python 인터페이스입니다. 인식 결과는 MCU에서 평가되는 표현식 문자열이며, 업로드된 바우카 프로그램에서 센서 통신과 제어를 실행합니다.

함수 19개

함수	기능
`begin_huskylens()`	`setup()`에 연결·지정 모드 전환 코드를 추가. 생성자 다음에 한 번 호출. 실패하면 `loop()`에서 재시도
`read_huskylens()`	`loop()`에 인식 결과 갱신 코드를 추가. 결과를 사용하는 코드보다 앞에 호출
`is_camera_connected()`	연결·모드 설정 및 최근 통신 상태 표현식. 새 통신 요청은 보내지 않음
`has_target()`	설정한 학습 ID·좌표 조건에 맞는 대상이 검출됐는지 확인
`get_target_id()`	선택한 대상의 학습 ID. 색 인식에서도 RGB 값이 아닌 ID
`get_target_x()`	사각형 중심 x. 선 추적에서는 화살표 끝점 x
`get_target_y()`	사각형 중심 y. 선 추적에서는 화살표 끝점 y
`get_target_width()`	사각형 너비, 단위는 픽셀. 선 추적에서는 사용하지 않음
`get_target_height()`	사각형 높이, 단위는 픽셀. 선 추적에서는 사용하지 않음
`get_arrow_x_origin()`	선 추적 화살표 시작점 x
`get_arrow_y_origin()`	선 추적 화살표 시작점 y
`get_arrow_x_target()`	선 추적 화살표 끝점 x
`get_arrow_y_target()`	선 추적 화살표 끝점 y
`debug_huskylens(interval_ms=250)`	지정 주기로 camera/target/id/x/y/w/h 출력. 기본 250ms, 9600 baud
`motor(left, right)`	대상 유효성을 검사하며 모터 제어. 속도 -255~255, 정수 또는 하나의 C++ 속도 표현식
`delay(ms)`	지연 코드를 추가하고 지연 중 오래된 결과에 대한 모터 정지 검사
`export_sketch(directory)`	지정 폴더에 같은 이름의 `.ino`를 생성하고 경로 반환

함수	기능
`compile_code(directory=..., fqbn=...)`	Arduino CLI로 컴파일만 수행하고 `.ino` 경로 반환
`upload_code(port=None, directory=..., fqbn=...)`	컴파일 후 업로드하고 `.ino` 경로 반환. `port=None`이면 자동 탐색

인식 모드 7개

`algorithm` 값	허스키렌즈 모드
`face_recognition`	Face Recognition / 얼굴 인식
`color_recognition`	Color Recognition / 색 인식
`object_tracking`	Object Tracking / 물체 추적
`object_recognition`	Object Recognition / 물체 인식
`line_tracking`	Line Tracking / 선 추적
`tag_recognition`	Tag Recognition / 태그 인식
`object_classification`	Object Classification / 물체 분류

생성자의 `algorithm`으로 모드를 지정하고 `begin_huskylens()`로 시작 시 적용합니다. 실행 중 모드를 바꾸는 별도의 Python 함수는 현재 제공하지 않습니다. 기본 주행·센서 전용 `StandaloneBowCar`의 모터 함수와 이 표의 허스키렌즈 대상 검사 기능을 구분하세요.

상세 함수 설명과 사용 예제 / 검증 기록 / 교육 예제 파일